Numbers and Representations - Part 2

K. Sridharan

Review

- In part 1, we discussed about different ways to represent binary numbers.

Beyond binary (base-2)

- Binary uses a base of 2.
- Frequently, we use other number systems for convenience where the base is different. For example, the decimal number system uses a base of 10.
- Computers use binary so we need to have a way of converting from decimal to binary (and back when required).

Decimal to binary conversion

$179 \div 2 = 89$ remainder 1 (LSB)

$\div 2 = 44$ remainder 1

$\div 2 = 22$ remainder 0

$\div 2 = 11$ remainder 0

$\div 2 = 5$ remainder 1

$\div 2 = 2$ remainder 1

$\div 2 = 1$ remainder 0

$\div 2 = 0$ remainder 1 (MSB)

$179_{10} = 10110011_2$

How do you verify this ?

- How do you show that $179_{10} = 10110011_2$?
- Question for students

How do you verify this ?

- We can show 10110011_2 is indeed the base-2 representation for 179 by using the weights of each position.

What about fractions ?

- Suppose we want to convert 0.6875. Then

$$
\begin{aligned}
0.6875 \times 2 &= 1 + 0.3750 & a_{-1} &= 1 \\
0.3750 \times 2 &= 0 + 0.7500 & a_{-2} &= 0 \\
0.7500 \times 2 &= 1 + 0.5000 & a_{-3} &= 1 \\
0.5000 \times 2 &= 1 + 0.0000 & a_{-4} &= 1
\end{aligned}
$$

So it is 0.1011 in base-2 (binary)

Verification

- Verify that 0.1011 is indeed the binary (base-2) representation for 0.6875.

Exercise for students.

Can extend to negative numbers too !

- If we are given a number like -2.5, we can find its binary representation (in 2's complement format). We will skip the details.

Reverse conversion: Binary to Decimal

- Recall unsigned binary slide (and the magnitude of the number). Examples below:

$$10011_2 = 1 \cdot 16 + 0 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 1 \cdot 1 = 19_{10}$$

$$100010_2 = 1 \cdot 32 + 0 \cdot 16 + 0 \cdot 8 + 0 \cdot 4 + 1 \cdot 2 + 0 \cdot 1 = 34_{10}$$

$$101.001_2 = 1 \cdot 4 + 0 \cdot 2 + 1 \cdot 1 + 0 \cdot 0.5 + 0 \cdot 0.25 + 1 \cdot 0.125 = 5.125_{10}$$

Other number systems

- For microprocessor programming, especially in assembly language (you will learn next semester), the hexadecimal (base-16) number system is useful.
- In the hexadecimal (hex) number system, we have digits from 0 to 9, then the alphabets A-F to represent numbers from 10 to 15.
- So we can write 16 bits compactly with 4 hex digits
- How do we convert from binary to hexadecimal (or hex) format ?

Binary to Hexadecimal

- Just divide the given binary number in groups of 4 bits starting from the least significant bit (LSB).

$$100011001110_2 = 1000\ 1100\ 1110_2 = 8CE_{16}$$

$$1110110111010101001_2 = 00011101\ 1011\ 1010\ 1001_2 = 1DBA9_{16}$$

Note we have added 0s to the left in the second number to keep the magnitude same

What about the converse ?

- To get the complete binary representation from hexadecimal, just write the binary representation for each hexadecimal character.
- If we can divide into groups of 4, why not divide into groups of 3 ? Yes, we can and it leads to the Octal number system.
- Once again, the benefit is in terms of a compact representation.

Octal number system

- We have numbers from 0 to 7 here. Examples follow.

$$100011001110_2 = 100\,011\,001\,110_2 = 4316_8$$

$$1110110111010100 1_2 = 011\,101\,101\,110\,101\,001_2 = 355651_8$$

When the total number of bits is not a multiple of 3, just add 0s to the left hand side.

What about binary numbers with fractional parts ?

- Just add zeros to the left and right of the given binary number (as appropriate: i.e., depending on the number of bits in the given number). Examples:

$$10.1011001011_2 = 010\,.\,101\;100\;101\;100_2 = 2.5454_8$$

$$= 0010\,.\,1011\;0010\;1100_2 = 2.B2C_{16}$$

Which is better – Hexadecimal or Octal ?

- Octal was of interest around 1980s since some minicomputers had their front-panel lights and switches arranged in groups of three.
- However, most computers today process bytes (i.e 8 bits of data), hence it is more useful to employ hexadecimal when we want compactness of representation.

Can we convert from hexadecimal to decimal ?

- Yes, we can use the weights directly and get the base-10 (decimal) representation.
  Examples:

$$1CE8_{16} = 1 \cdot 16^3 + 12 \cdot 16^2 + 14 \cdot 16^1 + 8 \cdot 16^0 = 7400_{10}$$

$$F1A3_{16} = 15 \cdot 16^3 + 1 \cdot 16^2 + 10 \cdot 16^1 + 3 \cdot 16^0 = 61859_{10}$$

How about the converse ?

- Given a decimal number, conversion to hexadecimal can be done by getting the binary equivalent first (of the decimal number) and then dividing into groups of 4.
- Is there a direct procedure that skips the conversion to binary first ? Yes, we will look at this next.

Decimal to hex directly

$3417 \div 16 = 213$ remainder 9 (least significant digit)

$\div 16 = 13$ remainder 5

$\div 16 = 0$ remainder 13 (most significant digit)

$3417_{10} = D59_{16}$

What about octal to decimal ?

- A procedure similar to the one for hexadecimal can be adopted.
- Can, in fact, extend to an octal number with a fractional part too. Example below:

$$436.5_8 = 4 \cdot 8^2 + 3 \cdot 8^1 + 6 \cdot 8^0 + 5 \cdot 8^{-1} = 286.625_{10}$$

What other representations exist ?

- Besides the representations we have discussed, there are a few others.
- One of them is called Binary-Coded-Decimal (BCD) representation. This is useful when we deal with unsigned numbers. In particular, each decimal digit (from 0 to 9) is represented by four bits. So 0 is represented as 0000 while 9 is represented as 1001

Advantages and limitations of BCD representation

- In a simple digit-oriented display, this way of representing each digit is okay.
- However, we are using only 10 of the 16 available 4-bit patterns. This means the remaining patterns such as 1010, 1111 etc. should not occur in logic circuits that operate on BCD operands. Further, circuits for arithmetic operations using BCD are complex (for eg., note that addition of two BCD digits is complicated when sum exceeds 9).

Textbook and Reference

- M. Mano and M. Ciletti, Digital Design, Pearson Education, 2018
- J. Wakerly, Digital Design – Principles & Practices, Pearson Education, 2008